RECUPERATORIO PARCIAL DISEÑO DE SOFTWARE 2023

TEORÍA

1. El patrón de diseño **FACTORY METHOD** define una interfaz para crear un objeto, pero deja que las subclases alteren el tipo de objetos que se crearan.

FACTORY METHOD (MÉTODO DE FABRICACIÓN)

Consiste en utilizar una clase constructora abstracta con unos cuantos métodos definidos y otros abstractos. Es una simplificación del Abstract Factory, en la que la clase abstracta tiene métodos concretos que usan algunos de los abstractos; según usemos una u otra hija de esta clase abstracta, tendremos uno u otro comportamiento.

- *Define una interfaz para crear un objeto, pero deja que sean las subclases quienes decidan qué clase instanciar. Permite que una clase delegue en sus subclases la creación de objetos.*

Este patrón debe ser utilizado cuando:

- Una clase no puede anticipar el tipo de objeto que debe crear y quiere que sus subclases especifiquen dichos objetos.
- Hay clases que delegan responsabilidades en una o varias subclases. Una aplicación es grande y compleja y posee muchos patrones creacionales.

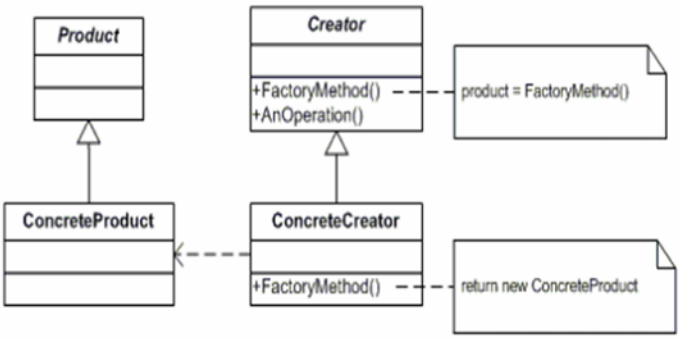
APLICABILIDAD

- Una clase **no puede prever** la clase de objetos que debe crear.
- Una clase quiere que sean sus **subclases** quienes **especifiquen los objetos** que ésta crea.
- Las clases **delegan la responsabilidad** en una de entre varias clases auxiliares, y queremos **localizar** qué subclase de auxiliar concreta es en la que se delega.

CONSECUENCIAS

1. Proporciona enganches para las subclases.
2. Conecta jerarquías de clases paralelas.

ESTRUCTURA



Creator: Declara el metodo FactoryMethod(). Se puede definir una implementación por defecto para él.
ConcreteCreator: Sobrescribe el método FactoryMethod(), devolviendo una instancia de ConcreteProduct
Product: Define la interfaz de objetos que crea el método FactoryMethod
ConcreteProduct: Implementa la interfaz del producto

2. El patrón de diseño **ADAPTER** permite que la interfaz de una clase existente se adapte a otra interfaz sin cambiar su código.

ADAPTER (ADAPTADOR)

Se utiliza para transformar una interfaz en otra, de tal modo que una **clase** que no pueda utilizar la primera haga uso de ella a través de la segunda.

- *Convierte la interfaz de una clase en otra interfaz que es la que esperan los clientes. Permite que cooperen las clases que de otra forma no podrían por tener interfaces incompatibles.*

Este patrón se debe utilizar cuando:

- Se quiere utilizar una clase que llame a un método a través de una interface, pero se busca utilizarlo con una clase que no implementa esa interface.
- Se busca determinar dinámicamente qué métodos de otros objetos llama un objeto.
- No quiere que el objeto llamado tenga conocimientos de la otra clase de objetos.

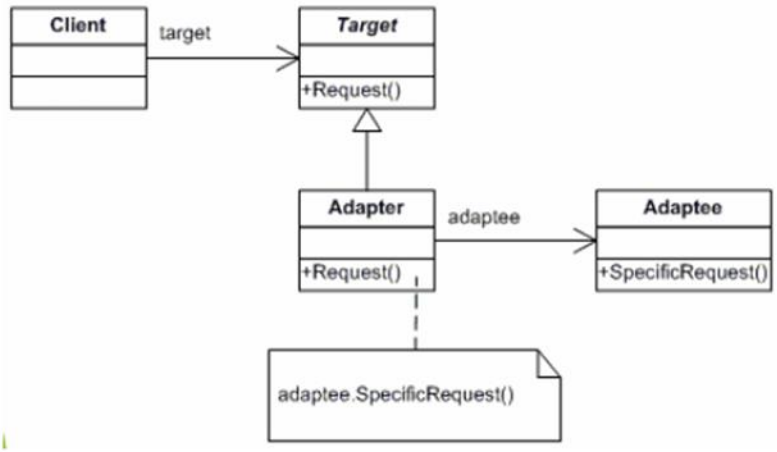
APLICABILIDAD

- Se quiere usar una clase existente y su interfaz no concuerda con la que necesita.
- Se quiere crear una clase reutilizable que coopere con clases no relacionadas o que no han sido previstas, es decir, clases que no tienen porqué tener interfaces compatibles.

CONSECUENCIAS

1. Adapta una clase Adaptable a Objetivo, pero se refiere únicamente a una clase Adaptable concreta. Por tanto, un adaptador de clases no nos servirá cuando lo que queremos es adaptar una clase y todas sus subclases.
2. Permite que Adaptador redefina parte del comportamiento de Adaptable, por ser Adaptador una subclase de Adaptable.
3. Introduce un solo objeto, y no se necesita ningún puntero de indirección adicional para obtener el objeto adaptado.

ESTRUCTURA



Target: Define una interfaz de dominio específico que el cliente utiliza
Adapter: Adapta interfaces de Adaptee y Target
Adaptee: Define una interfaz que existiendo necesita adaptación
Client: Utiliza objetos ajustándose a la interfaz ofrecida por Target

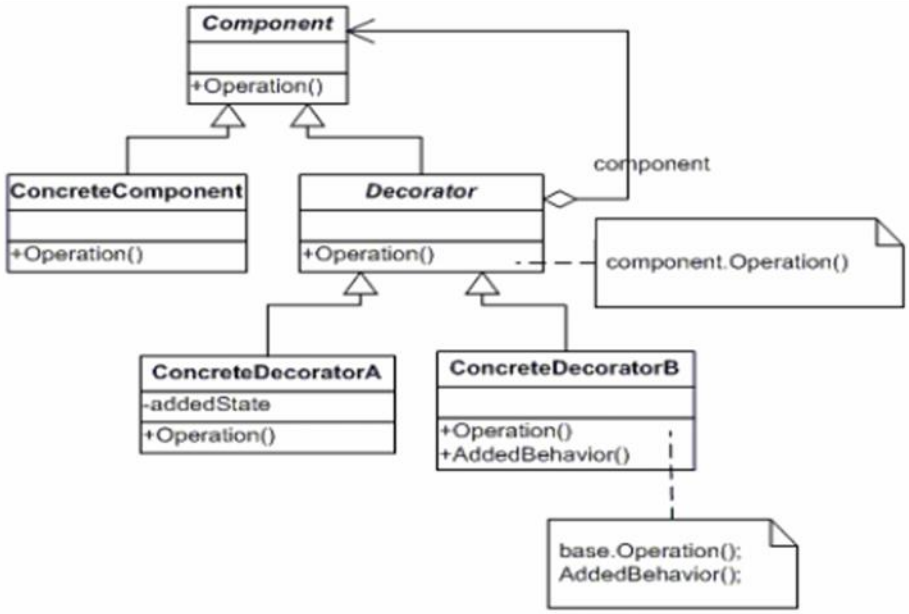
3. El patrón de diseño **DECORATOR** adjunta responsabilidades adicionales a un objeto de manera dinámica. Los decoradores proporcionan una alternativa flexible a la herencia para extender la funcionalidad.

DECORATOR (DECORADOR)

Asigna responsabilidades adicionales a un objeto dinámicamente, proporcionando una alternativa flexible a la herencia para extender la funcionalidad.

Este patrón se debe utilizar cuando:

- Hay una necesidad de extender la funcionalidad de una clase, pero no hay razones para extenderlo a través de la herencia.
- Se quiere agregar o quitar dinámicamente la funcionalidad de un objeto.



Component: Define la interfaz que ofrecen los objetos que poseen responsabilidades añadidas dinámicamente

ConcreteComponent: Define un objeto al que responsabilidades adicionales pueden serle añadidas

Decorator: Mantiene referencia a un objeto Component y define una interfaz que conforma el interfaz de Component

ConcreteDecorator: Añade las responsabilidades al Component

4. El principio de **INDEPENDENCIA FUNCIONAL** siguiere que una clase debe delegar la responsabilidad de otras clases siempre que sea posible para reducir la dependencia entre las clases.

8.3.7 Independencia funcional

El concepto de independencia funcional es resultado directo de la separación de problemas y de los conceptos de abstracción y ocultamiento de información. En escritos cruciales sobre el diseño de software, Wirth [Wir71] y Parnas [Par72] mencionan técnicas de mejora que promueven la independencia modular. Los trabajos posteriores de Stevens, Myers y Constantine [Ste74] dan solidez al concepto.

La independencia funcional se logra desarrollando módulos con funciones "miopes" que tengan "aversión" a la interacción excesiva con otros módulos. Dicho de otro modo, debe diseñarse software de manera que cada módulo resuelva un subconjunto específico de requerimientos y tenga una interfaz sencilla cuando se vea desde otras partes de la estructura del programa. Es lógico preguntar por qué es importante la independencia.

El software con modularidad eficaz, es decir, con módulos independientes, es más fácil de desarrollar porque su función se subdivide y las interfaces se simplifican (cuando el desarrollo es efectuado por un equipo hay que considerar las ramificaciones). Los módulos independientes son más fáciles de mantener (y probar) debido a que los efectos secundarios causados por el diseño o por la modificación del código son limitados, se reduce la propagación del error y es posible obtener módulos reutilizables. En resumen, la independencia funcional es una clave para el buen diseño y éste es la clave de la calidad del software.

La independencia se evalúa con el uso de dos criterios cualitativos: la cohesión y el acoplamiento. La *cohesión* es un indicador de la fortaleza relativa funcional de un módulo. El *acoplamiento* lo es de la independencia relativa entre módulos.

5. El principio de **ALTA COHESIÓN** sugiere que las clases deben tener un conjunto de responsabilidad altamente relacionadas y coherentes.

ALTA COHESIÓN

Problema: ¿Cómo mantener la complejidad dentro de límites manejables?

En el DOO, la **cohesión** es una medida de cuán **enfocados** están las **responsabilidades** de una **clase**.

Una alta cohesión caracteriza a las clases con responsabilidades estrechamente relacionadas para que no realicen un trabajo enorme.

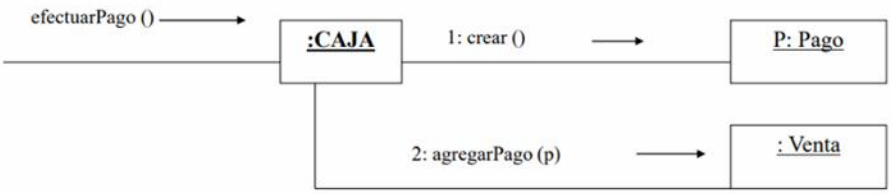
A menudo representan un alto grado de abstracción o han asumido responsabilidades que deberían haber delegado.

Una clase con **baja cohesión** hace muchas cosas no afines o un trabajo excesivo.

No conviene este tipo de clases, porque:

- Son difíciles de comprender.
- Son difíciles de reutilizar.
- Son difíciles de conservar.
- Son delicadas: las afectan constantemente los cambios.

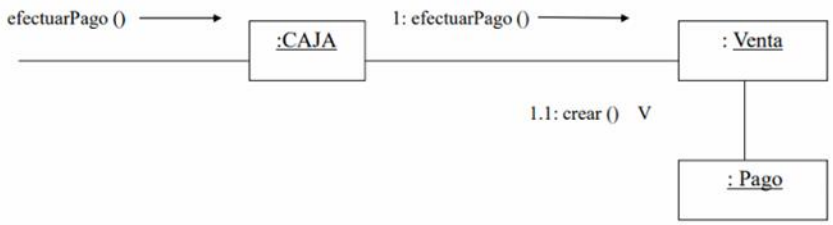
Solución: Asignar una responsabilidad de modo que la cohesión siga siento alta.



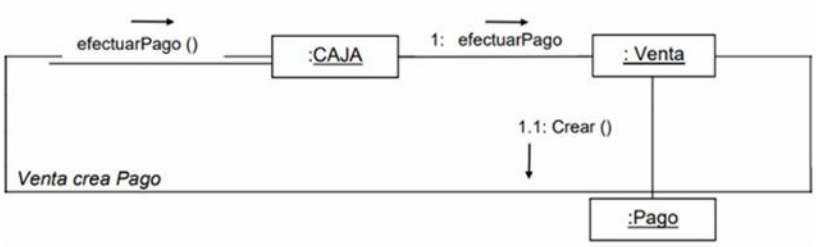
Esta asignación de responsabilidad coloca en la clase CAJA la generación del pago (patrón **Creador**)

Esto es aceptable. Pero si seguimos haciendo que la clase CAJA se encargue de efectuar algún trabajo con un número creciente de operaciones del sistema, se irá saturando con tareas y terminará por perder la cohesión.

En cambio, en el 2º ejemplo, se delega a la Venta la responsabilidad de **crear** el pago. Esto brinda un soporte a una **Alta Cohesión** y a un **Bajo Acoplamiento**.



El nivel de cohesión no puede ser considerado independientemente del **Experto** y del **Bajo Acoplamiento**.



Grados de cohesión

- **Muy baja Cohesión**: una **clase** es la **única responsable** de muchas cosas distintas en áreas funcionales diferentes (se deben dividir las responsabilidades en una familia de clases relacionadas).
- **Baja Cohesión**: una **clase** tiene la **responsabilidad exclusiva** de una **tarea compleja** dentro de una área funcional (dividir la clase en una familia de clases ligeras que compartan el trabajo).
- **Alta Cohesión**: una **clase tiene responsabilidades moderadas** en un área y colabora con las otras para realizar las tareas (supuesto ideal)
- **Cohesión Coincidentes**: Módulos múltiples tareas.
- **Cohesión Lógica:** Módulo con muchas tareas, pero en ejecución solo se ejecuta una.
- **Cohesión de Procedimiento:** Módulo con muchas secuencias de pasos.
- **Cohesión de Información:** Trabajan sobre los mismos datos.
- **Cohesión Funcional:** Módulo que ejecuta una única tarea.

Una **clase de Alta Cohesión** posee un número relativamente pequeño, con una importante funcionalidad relacionada y poco trabajo por hacer. **Colabora con otros objetos para compartir el esfuerzo si la tarea es grande.**

Una **clase** con mucha **Cohesión** es útil porque es bastante **fácil** darle **mantenimiento, entenderla y reutilizarla.**

El patrón Alta Cohesión presenta semejanzas con el mundo real. Sabemos que, si alguien asume demasiadas responsabilidades, no será eficiente (hay que **delegar**).

Beneficios:

- Mejora la claridad y la facilidad con que se entiende el diseño.
- Se simplifica el mantenimiento.
- A menudo se genera un Bajo Acoplamiento.
- Mayor capacidad de reutilización.

6. El concepto de **ITERACIÓN** en RUP se refiere a un ciclo corto y repetitivo dentro de una fase, donde se desarrolla, prueba y ajusta una porción del sistema.

PRINCIPIOS DE RUP

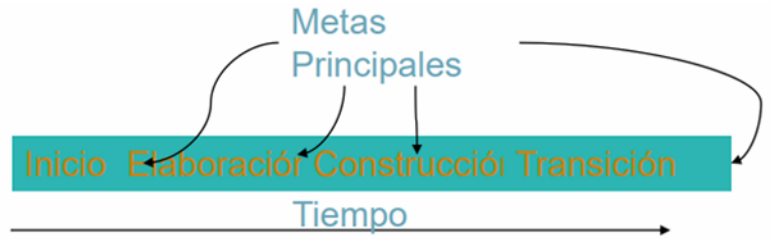
1. **ADAPTAR EL PROCESO**: El proceso deberá adaptarse a las necesidades del cliente, características del proyecto, regulaciones.
2. **EQUILIBRAR PRIORIDADES**: Los requisitos de los diversos participantes pueden ser diferentes, contradictorios o disputarse recursos limitados. Debe poder encontrarse un equilibrio que satisfaga los deseos de todos.
3. **COLABORACIÓN EN EL GRUPO**: El desarrollo de software no lo hace una única persona sino múltiples equipos. Debe haber una comunicación fluida para coordinar requisitos, desarrollo, evaluaciones, planes, resultados, etc.
4. **ITERACIÓN**: En cada iteración se analiza la opinión de los inversores, la estabilidad, calidad del producto y los riesgos involucrados.
5. **ABSTRACCIÓN**: Uso de conceptos reutilizables tales como patrones de diseño de software, lenguajes 4GL o frameworks.
6. **FOCO EN LA CALIDAD**: El control de la calidad no debe realizarse al final de cada iteración, sino en todos los aspectos de la producción. El aseguramiento de la calidad forma parte del proceso de desarrollo y no de un grupo independiente, también es una estrategia de desarrollo de software.

7. En RUP, la actividad **TRANSICIÓN** se centra en la entrega del sistema al usuario final y en recolección de comentarios para futuras iteraciones.

FASES EN RUP

FASES	OBJETIVOS
Inicio	Entender el problema
Elaboración	Entender la solución
Construcción	Tener la solución
Transición	Aceptar la solución

Inicio: Define el objetivo y alcance del proyecto, determinar los CU preliminares, estimación, costo, identificar riesgos
Elaboración: Plan del proyecto, especificación de características, arquitectura base, CU
Construcción: Construir el producto
Transición: Transición del producto a la comunidad del usuario



FASE DE TRANSICIÓN

- Hacer la transición final del producto de software al usuario
- **PRODUCTOS**
  - Liberaciones ejecutables de producto
  - "Pruebas beta" para validar el nuevo sistema vs las expectativas del usuario
  - Manuales de usuario actualizados
  - Documentación de desarrollo actualizada
- ¿Está el usuario satisfecho?
- Gastos reales de los recursos vs gastos previstos <- ¿Aceptables?

8. Larman presenta dos técnicas para identificar clases conceptuales, Lista de **CATEGORIAS DE CLASES CONCEPTUALES** e Identificación de **FRASES NOMINALES**.

Estrategias para identificar clases conceptuales

En las siguientes secciones se presentan dos técnicas:

1. Utilización de una lista de categorías de clases conceptuales.
2. Identificación de frases nominales.

Otra excelente técnica para el modelado del dominio es el uso de **patrones de análisis**, que son modelos de dominios parciales existentes creados por expertos, utilizando libros publicados como *Analysis Patterns* [Fowler96] y *Data Model Patterns* [Hay96].

Utilización de una lista de categorías de clases conceptuales

Comience la creación de un modelo del dominio haciendo una lista de clases conceptuales candidatas. La Tabla 10.1 contiene muchas categorías habituales que, normalmente, merece la pena tener en cuenta, aunque no en ningún orden particular de importancia. Los ejemplos se han extraído del dominio de las tiendas y las reservas de vuelos.

Tabla 10.1. Lista de categorías de clases conceptuales.

Categoría de clase conceptual	*Ejemplos*
objetos tangibles o físicos	*Registro* *Avion*
especificaciones, diseños o descripciones de la cosas	*EspecificacionDelProducto* *DescripcionDelVuelo*
lugares	*Tienda*
transacciones	*Venta, Pago* *Reserva*
líneas de la transacción	*LineaDeVenta*
roles de la gente	*Cajero* *Piloto*
contenedores de otras cosas	*Tienda, Lata* *Avion*
cosas en un contenedor	*Articulo* *Pasajero*
otros sistemas informáticos o electromecánicos externos al sistema	*SistemaAutorizacionPagoCredito* *ControlDeTraficoAereo*
conceptos abstractos	*Ansia* *Acrofobia*
organizaciones	*DepartamentoDeVentas* *CompañiaAerea*
hechos	*Venta, Pago, Reunion* *Vuelo, Colision, Aterrizaje*
procesos (normalmente *no* se representan como conceptos, pero podría ocurrir)	*VentaDeUnProducto* *ReservaUnAsiento*
reglas y políticas	*PoliticaDeReintegro* *PoliticaDeCancelación*
catálogos	*CatalogoDeProductos* *CatalogoDePiezas*
registros de finanzas, trabajo, contratos, cuestiones legales	*Recibo, LibroMayor, ContratoEmpleo* *RegistroMantenimiento*
instrumentos y servicios financieros	*LineaDeCredito* *Stock*
manuales, documentos, artículos de referencia, libros	*ListaDeCambiosDePreciosDiarios* *ManualReparaciones*

Descubrimiento de clases conceptuales mediante la identificación de frases nominales

Otra técnica útil (debido a su simplicidad) recomendada en [Abbot83] es el análisis lingüístico: identificar los nombres y frases nominales en las descripciones textuales de un dominio, y considerarlos como clases conceptuales o atributos candidatos.

> Se debe tener cuidado con este método; no es posible realizar una correspondencia mecánica de nombres a clases, y las palabras en lenguaje natural son ambiguas.

En cualquier caso, es otra fuente de inspiración. Los casos de uso en formato completo constituyen una descripción excelente a partir de la cual extraer este análisis. Por ejemplo, se puede utilizar el escenario actual del caso de uso *Procesar Venta*.

Escenario principal de éxito (o Flujo Básico):

1. El **Cliente** llega a un **terminal PDV** con **mercancías** y/o **servicios** que comprar.
2. El **Cajero** comienza una nueva **venta**.
3. El **Cajero** introduce el **identificador del artículo**.
4. El Sistema registra la **línea de la venta** y presenta la **descripción del artículo, precio** y **suma** parcial. El precio se calcula a partir de un conjunto de reglas de precios. El Cajero repite los pasos 3-4 hasta que se indique.
5. El Sistema presenta el total con los **impuestos** calculados.
6. El Cajero le dice al Cliente el total y solicita el **pago**.
7. El Cliente paga y el Sistema gestiona el pago.
8. El Sistema registra la **venta** completa y envía la información de la venta y el pago al sistema de **Contabilidad** externo (para la contabilidad y las **comisiones**) y al sistema de **Inventario** (para actualizar el inventario).
9. El Sistema presenta el **recibo**.
10. El Cliente se va con el recibo y las mercancías (si es el caso).

Extensiones (o Flujos Alternativos):

…

7a. Pago en efectivo:

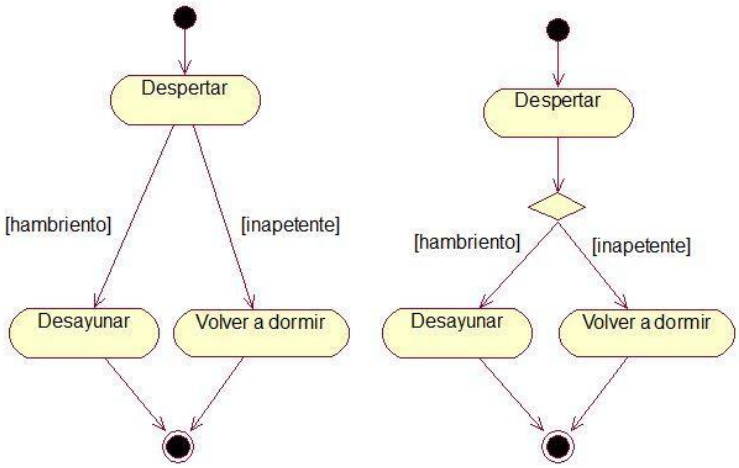
1. El Cajero introduce la **cantidad** de dinero **entregada** en efectivo.
2. El Sistema muestra la **cantidad** de dinero **a devolver** y abre el **cajón de caja**.
3. El Cajero deposita el dinero entregado y devuelve el cambio al Cliente.
4. El Sistema registra el pago en efectivo.

El modelo del dominio es una visualización de los conceptos del dominio y vocabulario relevantes. ¿Dónde se encuentran estos términos? En los casos de uso. Por tanto, constituyen una fuente rica a explorar mediante la identificación de frases nominales.

Algunas de las frases nominales son clases conceptuales candidatas, algunas podrían hacer referencia a clases conceptuales que se ignoran en esta iteración (por ejemplo, "Contabilidad" y "comisiones"), y algunas podrían ser atributos de las clases conceptuales. Por favor, diríjase a la siguiente sección y al capítulo sobre los atributos, para obtener los consejos que permiten diferenciar entre los dos.

Un punto débil de este enfoque es la imprecisión del lenguaje natural; frases nominales diferentes podrían representar la misma clase conceptual o atributo, entre otras ambigüedades. Sin embargo, se recomienda que se combine con la técnica la *Lista de Categorías de Clases Conceptuales*.

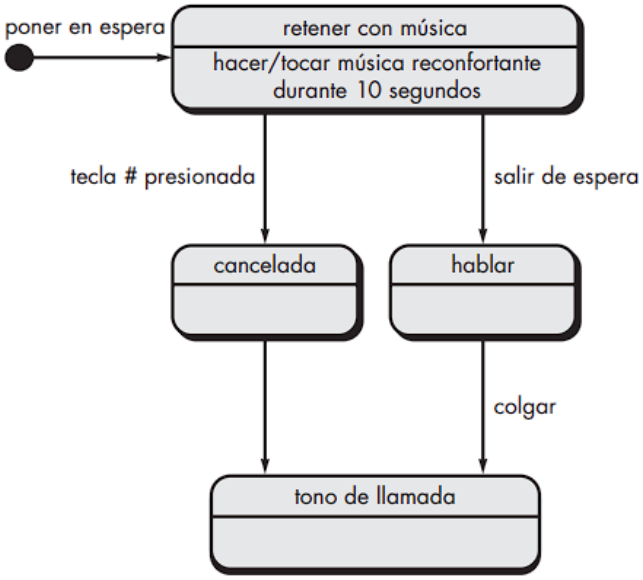
9. Seleccione la opción correcta:



- Diagrama A y B son correctos
- Diagrama A y B son incorrectos
- Diagrama A correcto y B incorrecto
- Diagrama B correcto y A incorrecto

10. Seleccione las opciones correctas:

La siguiente imagen usa los estados para la operación de un teléfono:



- La llamada pasa al estado retener con música (durante 10 segundos).
- El estado “hablar” se comporta como un estado normal de no actividad.
- No se puede presionar # antes de finalizar los 10 segundos de la música
- Si no se presiona la tecla # cuando la llamada está en el estado retener con música, la llamada transita hacia el estado cancelado e inmediatamente al estado tono de llamada.

PRACTICA

Una red de gimnasios, solicita un sistema de software que gestione usuarios, pagos, rutinas y brinde funcionalidades útiles para gimnasio y para sus clientes. Los requerimientos más relevantes son:

Gestión de Usuarios

- Registro de nuevos usuarios con información personal (Fecha de nacimiento, Genero, Dirección, Número de teléfono contacto, Correo electrónico, etc.).
- Registro de Contacto de Emergencia (Nombre del contacto de emergencia, Número de teléfono del contacto emergencia, Relación con el usuario)
- Perfiles de usuarios con detalles específicos sobre su condición física, objetivos de entrenamiento y restricciones médicas.
- Posibilidad de asignar un entrenador personal a cada usuario.
- Tipo de ejercicio preferido (cardio, pesas, clases, etc.)

Gestión de Entrenadores

- Gestión de entrenadores (Nombre completo, Información de contacto: Teléfono, correo electrónico. Detalles certificaciones y especialidades)
- Horarios de disponibilidad semana de cada entrenador
- Asignación de Clientes.
- Capacidad para organizar y gestionar sesiones de entrenamiento.

Programación de Rutinas

- Creación y asignación de rutinas de entrenamiento por parte de los entrenadores. Podrán seleccionar la rutina: musculación / aeróbico / clase y adicionar información de acuerdo al tipo de rutina.
- Acceso para que los usuarios vean y sigan sus rutinas personalizadas.
- Registro de progreso en las rutinas.
- Registro de asistencia.

Ejemplo de Rutinas

Rutina: Musculación / Aeróbico / Clase
DIA: 13-11-2023

Musculo	Ejercicio	Serie	Repeticiones	Peso	Estado
Pierna	Sentadilla	3	8	20	Hecho
Pecho	Press Banca	2	12	20	Pendiente
Espalda	Peso Muerto	3	10	25	Pendiente
Abdomen	Rueda Abdominal	3	8	-	Pendiente

Rutina: Aeróbico
DIA: 13-11-2023

Ejercicio	Minutos	Estado
Caminar	15	Hecho
Correr	25	Pendiente
Bicicleta	25	Pendiente

Rutina: Clase (CrossFit, GAP, HIIT, MMA, Yoga, Bodypump, indoor, boxeo)
DIA: 13-11-2023

Disciplina	Minutos	Estado
CrossFit	60	Hecho

Gestión de Membresías y Pagos:

- Registro y seguimiento de membresías.

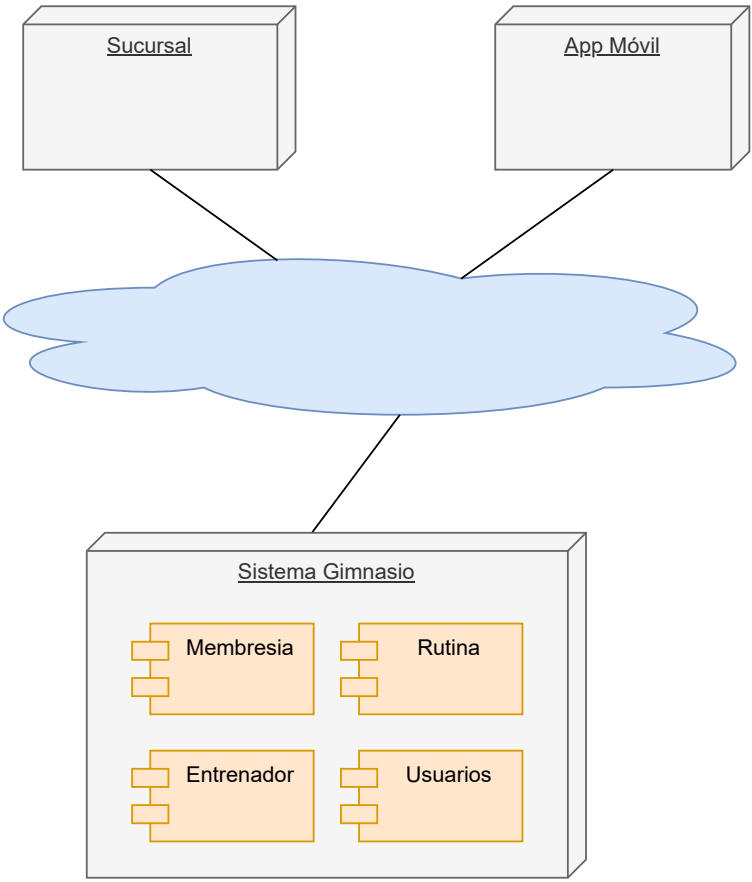
- Información de Membresía:
  - Tipo de Membresía (Mensual, Anual, etc.)
  - Fecha de inicio de la membresía
  - Fecha de vencimiento de la membresía
  - Estado de la membresía (activo, vencido, congelado, etc.)
  - Historial de pagos y transacciones
- Alertas automáticas para recordar a los usuarios cuando su membresía esta por vencer.

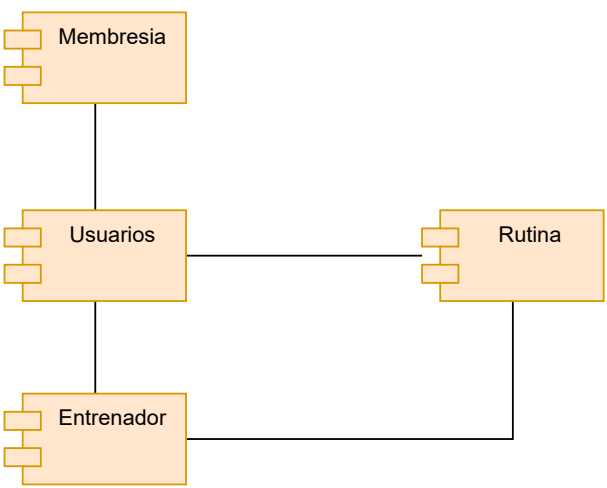
NOTA: En esta instancia NO contempla el pago, solamente contempla si está pago o si está pendiente.

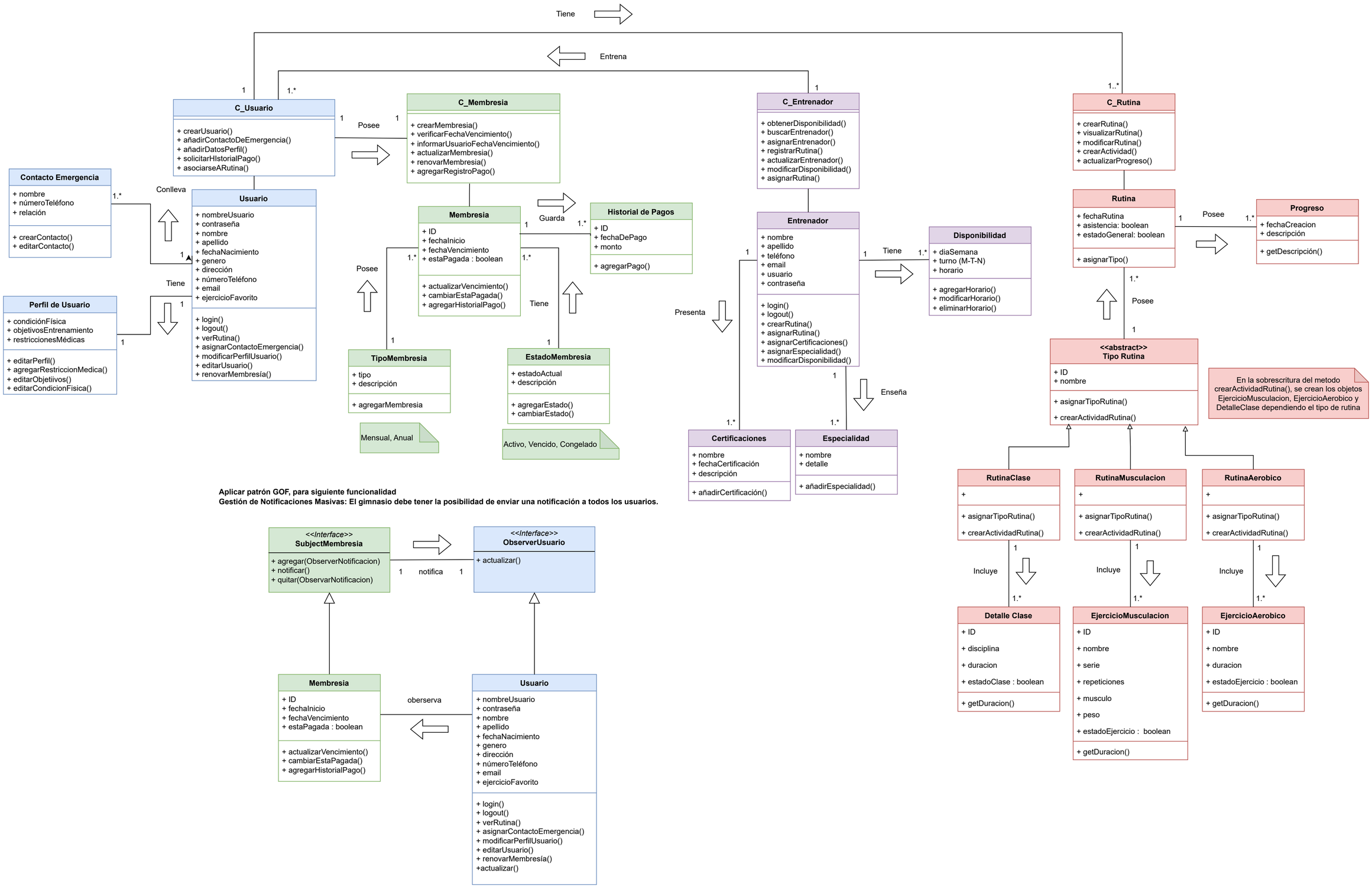
Se pide:

1. Diagrama de Clases de Diseño COMPLETO
2. Aplicar patrón GOF, para siguiente funcionalidad
   Gestión de Notificaciones Masivas: El gimnasio debe tener la posibilidad de enviar una notificación a todos los usuarios.
3. Diagrama de Secuencia para el siguiente escenario.

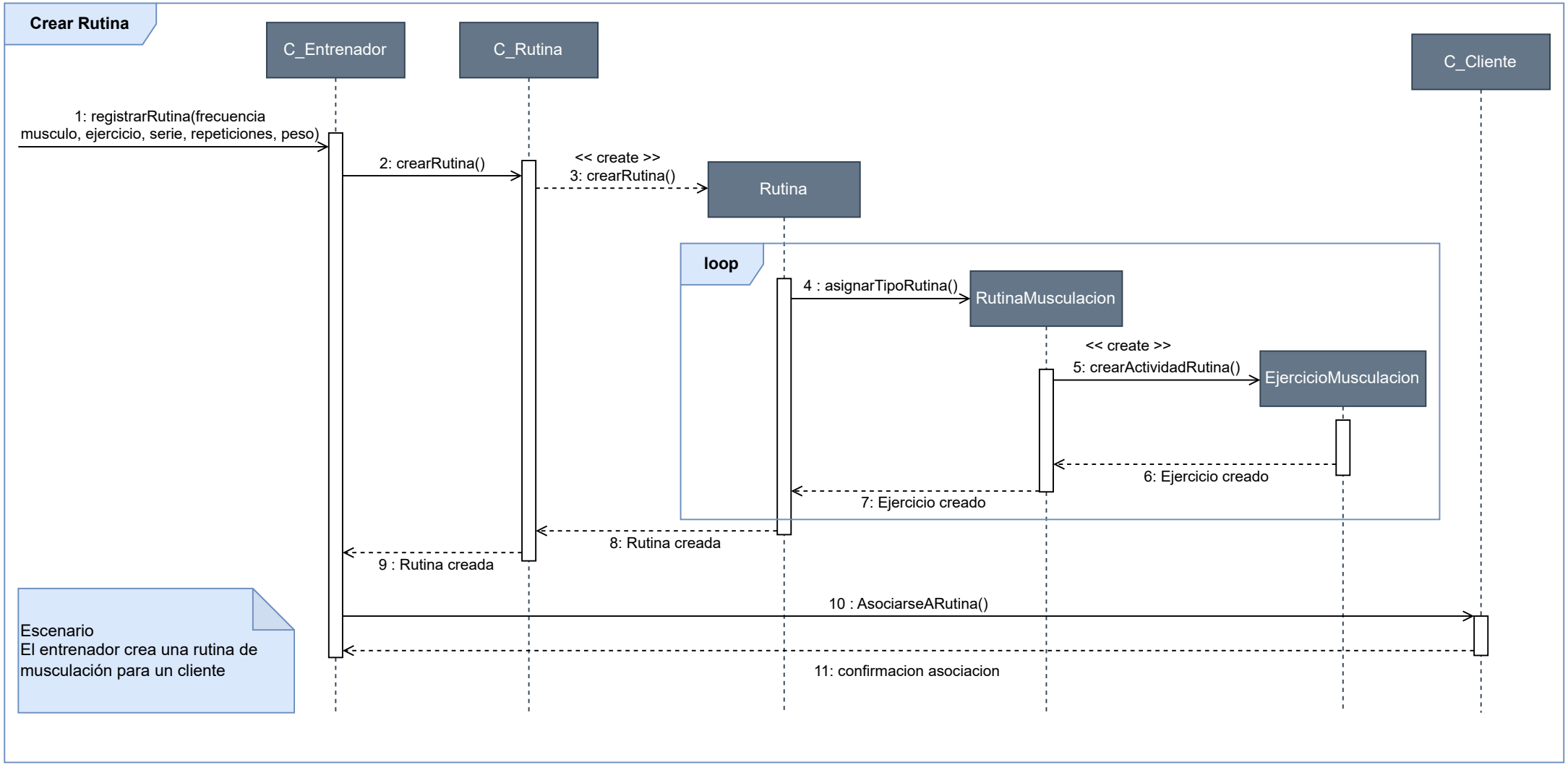
	Objetivo: Crear una rutina de musculación personalizadas para un cliente		
	Precondiciones • El entrenador ha iniciado sesión en el sistema. • Existen clientes registrado en el sistema • El entrenador ya se encuentra con el cliente seleccionado.		
	ENTRENADOR		**SISTEMA**
1	El Entrenador selecciona la opción de "Crear Rutina" de Musculación	**1**	
2		**2**	El Sistema presenta una interfaz para definir el tipo de rutina
3	El Entrenador selecciona la opción Musculación	**3**	
4		**4**	El Sistema presenta una interfaz para la creación de la rutina.
5	El entrenador ingresa **Frecuencia** Semanal: Cantidad de veces para realizar la rutina **Musculo**, Ejercicio, Serie, Repeticiones, peso (opcional): El peso que se debe utilizar para el ejercicio (este campo es opcional y puede dejarse en blanco). Selecciona Guardar	**5**	
6		**6**	El Sistema registra la rutina y muestra la rutina
7	El Entrenador revisa la rutina y cierra la rutina en el Sistema	**7**	
	Se repite el paso para crear rutina		
*	**Extensiones** • Si el entrenador decide abandonar la creación de la rutina en cualquier momento, se proporciona la opción de descartar la rutina sin guardar cambios.		
	Postcondiciones • La rutina de musculación ha sido creada y asociada al cliente seleccionado. • El cliente puede acceder a su rutina a través de su cuenta en el sistema.		


Sucursal
App Móvil
Sistema Gimnasio
Membresia
Rutina
Entrenador
Usuarios


Membresia
Usuarios
Rutina
Entrenador



Aplicar patrón GOF, para siguiente funcionalidad
Gestión de Notificaciones Masivas: El gimnasio debe tener la posibilidad de enviar una notificación a todos los usuarios.


Crear Rutina
C_Entrenador
C_Rutina
C_Cliente
1: registrarRutina(frecuencia
musculo, ejercicio, serie, repeticiones, peso)
2: crearRutina()
<< create >>
3: crearRutina()
Rutina
loop
4 : asignarTipoRutina()
RutinaMusculacion
<< create >>
5: crearActividadRutina()
EjercicioMusculacion
6: Ejercicio creado
7: Ejercicio creado
8: Rutina creada
9 : Rutina creada
10 : AsociarseARutina()
11: confirmacion asociacion
Escenario
El entrenador crea una rutina de
musculación para un cliente